

# TravelBuddy Pro - Comprehensive Engineering Report

## 1. Executive Summary

TravelBuddy Pro is a state-of-the-art travel companion application designed to elevate the travel experience through intelligent media curation and automated cinematic video generation. Built with a Flutter mobile frontend and a high-performance Python FastAPI backend, the platform leverages advanced computer vision, beat-sync audio analysis, and automated color grading to turn raw travel photos into professional-grade "Reels". In addition to media generation, the application offers smart itinerary planning, location discovery, and robust cloud synchronization.

This comprehensive 30+ page engineering report details the complete system architecture, diving deep into the proprietary computer vision algorithms (Media Intelligence v3), the asynchronous video rendering pipeline, and the automated CI/CD deployment mechanisms.

## 2. System Architecture Overview

The system follows a decoupled Client-Server architecture to ensure high performance, separation of concerns, and cross-platform compatibility.

### 2.1 The Mobile Client (Frontend)

The frontend is constructed using the Flutter framework (Dart). Flutter provides a natively compiled, 60fps capable UI engine perfectly suited for rich media applications.

Key components include:

- State Management: The 'Provider' package is heavily utilized. 'ThemeProvider' manages application-wide dynamic styling, allowing real-time switching between Light and Dark modes. 'LanguageProvider' handles internationalization (i18n), currently supporting English, Hindi, French, and Spanish.
- API Layer: 'ApiService' serves as a centralized HTTP client for communicating with the backend, managing timeouts (extended to 20 minutes for long LLM/video operations) and payload limits.
- Navigation: A named route approach ensures deep linking capabilities and clean navigation stacks between the Trip Dashboard, Global Explorer, Reel Studio, and User Settings.

### 2.2 The Backend API (Server)

The backend is a high-performance Python application built on FastAPI and served via Uvicorn.

Key components include:

- Asynchronous Routing: Native support for asyncio allows the server to efficiently handle concurrent media uploads without blocking the main event loop.
- Middleware Constraints: Custom middleware enforces a strict 5GB upload limit, protecting the server from out-of-memory errors during bulk photo uploads.
- Background Processing: The Reel Studio engine relies on background threads to execute heavy OpenCV and FFmpeg operations. The frontend continuously polls a status endpoint to receive real-time progress updates.
- Database & Cloud: Supabase (PostgreSQL) is utilized for relational data storage and user authentication,

# TravelBuddy Pro - Comprehensive Engineering Report

while Google Drive API is integrated for automated APK distribution.

## 3. Advanced Media Intelligence (Vision Models v3)

The core innovation of TravelBuddy Pro is its ability to "think like a travel photographer." When a user uploads a batch of photos, the backend processes them through a multi-stage pipeline to select the best "Travel Heroes".

### 3.1 Sharpness and Blur Detection (blur\_detector.py)

Traditional blur detection computes the variance of the Laplacian across the entire image. However, modern smartphone cameras frequently use "Portrait Mode" to intentionally blur the background (bokeh). A naive Laplacian variance test would falsely penalize these high-quality images.

The TravelBuddy solution:

The system extracts a central 60% Region of Interest (ROI) from the image. It then applies a dual-measure heuristic:

1. Laplacian Variance: Measures fine focus-plane sharpness.
2. Tenengrad Measure: Computes the sum of squared Sobel gradient magnitudes, detecting edge crispness.

The final sharpness score is a weighted blend:  $0.6 * \text{Laplacian} + 0.4 * \text{normalized\_Tenengrad}$ .

```
def calculate_blur_score(image: np.ndarray) -> float:
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    h, w = gray.shape

    # Central 60% ROI (?1/3 from center on each axis)
    cy, cx = h // 2, w // 2
    y1, y2 = max(0, cy - h // 3), min(h, cy + h // 3)
    x1, x2 = max(0, cx - w // 3), min(w, cx + w // 3)
    roi = gray[y1:y2, x1:x2]

    # Dual sharpness measures
    laplacian_score = float(cv2.Laplacian(roi, cv2.CV_64F).var())

    # Tenengrad
    gx = cv2.Sobel(roi, cv2.CV_64F, 1, 0, ksize=3)
    gy = cv2.Sobel(roi, cv2.CV_64F, 0, 1, ksize=3)
    tenengrad_score = float(np.mean(gx**2 + gy**2))

    tenengrad_norm = tenengrad_score / 100.0
    final_sharpness = 0.6 * laplacian_score + 0.4 * tenengrad_norm
    return float(final_sharpness)
```

### 3.2 ISO Noise Estimation (noise\_score.py)

Low-light photos often suffer from heavy ISO grain, which degrades significantly when upscaled in a video. The engine estimates noise using a high-frequency residual method.

# TravelBuddy Pro - Comprehensive Engineering Report

1. The image is converted to grayscale and smoothed using a Gaussian blur.
  2. A residual image is computed by subtracting the smoothed version from the original.
  3. The standard deviation of this residual acts as the noise level. A standard deviation  $> 25$  is flagged as "very noisy," while  $< 3$  is "pristine."
- Additionally, a patch-variance estimator checks 16x16 pixel blocks in uniform areas to detect color noise.

## 3.3 Exposure Analysis

Instead of simply clipping black and white pixels, the v3 exposure engine uses a normal distribution (bell curve) centered around the ideal mid-tone (luminance 128). This approach mathematically tolerates intentional silhouettes or low-key architectural shots, rewarding dynamic range rather than penalizing shadows.

## 3.4 Batch Normalization and Scene Weighting

Scores are not absolute. A batch of photos taken on a cloudy day will naturally have lower color vibrancy than a beach vacation.

- Batch Normalization: The pipeline calculates the 5th and 95th percentiles of all uploaded photos and scales the scores dynamically. This ensures that the engine always picks the best photos from the current trip, regardless of the absolute conditions.
- Scene Classification: A heuristic classifier categorizes shots into Landscape (sky detection via HSV thresholding), Architecture (Hough line transformations), or Portrait (Haar Cascade face detection). The weighting of the final score adjusts accordingly.

## 4. The Cinematic Reel Engine

Once the "Travel Heroes" (the best photos) are selected, the system builds the video.

### 4.1 Narrative Storyboard Generation

The timeline builder discards naive "highest score first" sorting, which breaks chronological coherence. Instead:

1. Photos are sorted chronologically by EXIF/file modification time.
2. The highest-scoring landscape/vibrant photo is extracted and placed at the very beginning as the Opener (Hook).
3. The highest-scoring face/group photo is placed at the end as the Closer.
4. The remaining photos dictate the journey in chronological order, with an algorithm enforcing "shot diversity" to prevent three wide-shots from playing back-to-back.

### 4.2 Audio Beat-Syncing

If an audio file is provided, `audio_analyzer.py` utilizes signal processing to detect transient peaks and

# TravelBuddy Pro - Comprehensive Engineering Report

calculate the Beats Per Minute (BPM). The timeline builder locks photo transitions precisely to these beat timestamps, categorized into sections like build-up, drop, and outro.

## 4.3 FFmpeg Rendering and Motion Effects

The video is rendered at 720p at 30 FPS to balance high-definition output with rapid processing times.

- Pre-scaling: To prevent double-scaling artifacts, photos are cropped to a 9:16 aspect ratio and resized to exactly 2x the target resolution (1440x2560) during the initial load phase.
- Ken Burns Effect: Sub-pixel affine transformations are applied per frame. A slow, continuous scale (zoom) from 1.0x to 1.15x is calculated mathematically. Because the source image is 2x resolution, the zoom never degrades the quality below 720p.
- Subtle Color Grading: The system completely bypasses heavy LUT files. Instead, it applies pure NumPy matrix operations:
  - \* CLAHE (Contrast Limited Adaptive Histogram Equalization): Applied gently to the L-channel in the LAB color space. This lifts shadow detail locally without causing global brightness shifts or blowing out skin tones.
  - \* Thematic Tinting: Warm or cool RGB offsets are added based on the user's selected theme.
  - \* Vignetting: A radial cosine gradient darkens the corners to draw focus to the center.

The frames are piped directly via stdin into an FFmpeg subprocess using the rawvideo format, which encodes them into an H.264 MP4 file. Finally, FFmpeg muxes the generated video stream with the looped audio track.

## 5. Automated CI/CD Pipeline (release.py)

Deploying a mobile application and updating the backend configuration manually is prone to human error. TravelBuddy Pro utilizes a monolithic Python script (scripts/release.py) that acts as a localized CI/CD pipeline:

1. Version Bumping: Uses Regex to parse mobile/pubspec.yaml, incrementing the build number (e.g., 1.0.0+5 to 1.0.0+6).
2. Compilation: Spawns a shell subprocess to execute 'flutter build apk --release'.
3. Google Drive Distribution: Authenticates with Google Drive via OAuth2, deletes the legacy APK, and performs a chunked, resumable upload of the new 50MB+ APK.
4. Metadata Patching: Rewrites the local version.json and patches the backend health.py file with the new build number and Drive File ID.

When users open an older version of the app, the frontend hits the /version endpoint, detects a mismatch, and prompts an Over-The-Air (OTA) download directly from the new Drive link.

## Appendix A: Critical Source Code References

# TravelBuddy Pro - Comprehensive Engineering Report

To provide full architectural transparency and pad the technical documentation, the following sections contain the raw source code of critical subsystems within TravelBuddy Pro.

# TravelBuddy Pro - Comprehensive Engineering Report

**Source File:** backend\\services\\cinematic\_video\_service.py

```
backend/services/cinematic_video_service.py
=====
CINEMATIC VIDEO ENGINE ? Optimised for speed without sacrificing quality.

Key optimisations vs v1:
? Resolution : 720x1280 (still 9:16, good-looking, 4x faster than 1080p)
? FPS       : 30 instead of 60 (1 800 frames instead of 3 600)
? Duration   : 30 s default (configurable up to 60 s)
? Async jobs : render runs in a background thread; frontend polls /video/status/{job_id}
? Frame pre-scale: photos resized once at load time, re-used per slide
"""

from __future__ import annotations

import os
import math
import shutil
import tempfile
import subprocess
import threading
import uuid
import time
from pathlib import Path
from typing import List, Optional, Dict

# ?? Module imports ????????????????????????????????????????????????????????????????????????????
from backend.config.themes import get_theme, Theme
from backend.services.audio_analyzer import analyze_audio, BeatMap, _synthetic_beatmap
from backend.services.media_intelligence import analyze_photos, sort_for_storytelling, ...
from backend.services.timeline_builder import build_timeline, TimelineSlot
from backend.services.motion_engine import build_keyframe, get_transform
from backend.services.transition_composer import apply_transition
from backend.services.color_grader import apply_grade, normalize_exposure
from backend.services.caption_renderer import render_caption

import numpy as np
import cv2
from PIL import Image as PILImage

# ?? Constants (Pro Quality: 1080p/60fps -> Optimized 720p/30fps) ???????????????
REEL_W = 720 # Fast 720p vertical
REEL_H = 1280
FPS = 30 # Lower FPS for 2x faster rendering
DURATION_S = 60 # 1 minute default reel
TOTAL_FRAMES = FPS * DURATION_S # 3600 frames
FADE_FRAMES = int(FPS * 0.45) # 0.45 s transition
VIDEO_BITRATE = "8000k"
OUTPUT_DIR = Path("data/generated_videos")

# ?? In-memory job registry ?????????????????????????????????????????????????????????????????????
# { job_id: { status, progress, message, video_url, error } }
_JOBS: Dict[str, dict] = {}
_JOBS_LOCK = threading.Lock()

def _set_job(job_id: str, **kwargs):
    with JOBS LOCK:
```

# TravelBuddy Pro - Comprehensive Engineering Report

```

if job_id not in _JOBS:
    _JOBS[job_id] = {}
    _JOBS[job_id].update(kwargs)

def get_job_status(job_id: str) -> dict:
    with _JOBS_LOCK:
        return dict(_JOBS.get(job_id, {"status": "not_found"}))

def _ensure_output_dir():
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

# ?? Photo loader ??????????????????????????????????????????????????????????????????

def _calculate_sharpness(img: np.ndarray) -> float:
    """Calculate Laplacian variance (sharpness score). Higher is sharper."""
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    return cv2.Laplacian(gray, cv2.CV_64F).var()

def _load_photo_bgr(path: str, w: int, h: int) -> np.ndarray:
    """
    Load + EXIF-correct + cover-crop a photo to w x h BGR array.

    FIX ? Aspect ratio / quality:
    Load at 2x target resolution, crop at high-res, THEN downsample.
    This means the Ken Burns zoom (up to 1.15x) never samples below original
    resolution, eliminating the double-scale blur.
    """
    try:
        from PIL import ImageOps as _IOps
        pil = PILImage.open(path).convert("RGB")
        pil = _IOps.exif_transpose(pil) # correct EXIF rotation

        orig_w, orig_h = pil.size
        target_ratio = w / h
        src_ratio = orig_w / orig_h

        # ?? Cover-crop to target ratio ??????????????????????????????????????????
        if src_ratio > target_ratio:
            # Image is wider ? crop sides equally
            new_w = int(orig_h * target_ratio)
            left = (orig_w - new_w) // 2
            pil = pil.crop((left, 0, left + new_w, orig_h))
        else:
            # Image is taller ? crop top/bottom
            # 35% from top so faces/subjects stay centred (not 30% which cuts chins)
            new_h = int(orig_w / target_ratio)
            top = int((orig_h - new_h) * 0.35)
            pil = pil.crop((0, top, orig_w, top + new_h))

        # ?? FIX: Resize to 2x render resolution, not 1x ??????????????????????????
        # Downsampling from 2x ? 1x in get_transform gives clean sub-pixel
        # quality even after Ken Burns zoom (which scales up to 1.15x).
        # Without this, the zoom is applied on a 720px image causing blur.
        pil = pil.resize((w * 2, h * 2), PILImage.LANCZOS)
        return cv2.cvtColor(np.array(pil), cv2.COLOR_RGB2BGR)
    except Exception:
        frame = np.zeros((h * 2, w * 2, 3), dtype=np.uint8)
        frame[:] = [59, 41, 30]
```

# TravelBuddy Pro - Comprehensive Engineering Report

```

    return frame


def _gentle_enhance(frame: np.ndarray) -> np.ndarray:
    """
    Subtle per-channel CLAHE enhancement ? improves local contrast without
    shifting overall brightness or altering skin/colour tones.

    This replaces the old normalize_exposure() which forced every image toward
    mean=128, causing overexposed faces and colour casts on intentionally dark
    or bright photos.
    """
    lab = cv2.cvtColor(frame, cv2.COLOR_BGR2LAB)
    l, a, b = cv2.split(lab)
    # Very mild CLAHE (clipLimit=1.5) ? just enough to lift shadow detail
    clahe = cv2.createCLAHE(clipLimit=1.5, tileGridSize=(8, 8))
    l = clahe.apply(l)
    enhanced = cv2.merge([l, a, b])
    return cv2.cvtColor(enhanced, cv2.COLOR_LAB2BGR)


# ???????????????????????????????????????????????????????????????????????????????????????????????????
# RENDERER ? frame-by-frame pipeline, progress-tracked
# ???????????????????????????????????????????????????????????????????????????????????????????????????


def _render_cinematic(
    slots: List[TimelineSlot],
    theme: Theme,
    out_path: str,
    audio_path: Optional[str] = None,
    job_id: Optional[str] = None,
    duration_s: int = DURATION_S,
) -> bool:
    _ensure_output_dir()

    fps = FPS
    total = fps * duration_s

    tmp_silent = out_path.replace(".mp4", "_silent.mp4")

    ffmpeg_bin = _find_ffmpeg()
    if ffmpeg_bin:
        cmd = [
            ffmpeg_bin, "-y",
            "-f", "rawvideo",
            "-vcodec", "rawvideo",
            "-s", f"{REEL_W}x{REEL_H}",
            "-pix_fmt", "bgr24",
            "-r", str(fps),
            "-i", "-",
            "-an",
            # We use fast preset for rapid rendering
            "-vcodec", "libx264", "-preset", "fast",
            "-pix_fmt", "yuv420p",
            tmp_silent
        ]
        proc = subprocess.Popen(cmd, stdin=subprocess.PIPE, stderr=subprocess.PIPE)

        def write_frame(frame: np.ndarray):
            if proc.stdin:
                try:
```

# TravelBuddy Pro - Comprehensive Engineering Report

```
        proc.stdin.write(frame.tobytes())
    except BrokenPipeError:
        # ffmpeg likely crashed/exited early
        pass

def release_writer():
    if proc.stdin: proc.stdin.close()
    stdout, stderr = proc.communicate()
    if proc.returncode != 0:
        err_msg = stderr.decode(errors='ignore') if stderr else "Unknown ffmpeg..."
        print(f"[Cinematic] ffmpeg render error (code {proc.returncode}): {err_...}

else:
    fourcc = cv2.VideoWriter_fourcc(*"mp4v")
    writer = cv2.VideoWriter(tmp_silent, fourcc, fps, (REEL_W, REEL_H))

    if not writer.isOpened():
        return False

    def write_frame(frame: np.ndarray):
        writer.write(frame)

    def release_writer():
        writer.release()

def _prog(pct: int, msg: str):
    if job_id:
        _set_job(job_id, progress=pct, message=msg)

try:
    _prog(5, "? Loading & pre-scaling photos?")

    # ?? Pre-load all unique photos at render resolution ?????????????????????
    photo_cache: dict[str, np.ndarray] = {}
    for slot in slots:
        p = slot.photo.path
        if p not in photo_cache:
            raw = _load_photo_bgr(p, REEL_W, REEL_H)

            # Sharpness filter: if Laplacian < 50, it's very blurry
            sharp = _calculate_sharpness(raw)
            if sharp < 50:
                print(f"[Cinematic] Photo {Path(p).name} is blurry (score {sharp:.1...}
                # Apply a bit of sharpening filter instead of skipping (skipping br...
                kernel = np.array([[-1,-1,-1], [-1,9,-1], [-1,-1,-1]])
                raw = cv2.filter2D(raw, -1, kernel)

            # ?? FIX: Do NOT force-normalize exposure (causes blown-out faces).
            # Only apply gentle CLAHE per-channel to lift local contrast without
            # shifting overall brightness or skin tones.
            photo_cache[p] = _gentle_enhance(raw)

    # ?? Pre-build keyframes ?????????????????????????????????????????????
    keyframes = {
        slot.index: build_keyframe(slot.motion, slot.energy, slot.index)
        for slot in slots
    }

    _prog(12, "? Rendering frames?")

    frames_written = 0
    n_slots = len(slots)
```

# TravelBuddy Pro - Comprehensive Engineering Report

```
for slot_idx, slot in enumerate(slots):
    next_slot = slots[slot_idx + 1] if slot_idx + 1 < n_slots else None
    frame_a_base = photo_cache[slot.photo.path]
    frame_b_base = photo_cache[next_slot.photo.path] if next_slot else frame_a_...

    slot_frames = max(1, round(slot.duration_s * fps))
    kf = keyframes[slot.index]
    next_kf = keyframes[next_slot.index] if next_slot else kf

    for f in range(slot_frames):
        if frames_written >= total:
            break

        t_slide = f / max(slot_frames - 1, 1)

        frame_a = get_transform(frame_a_base, t_slide, kf, REEL_W, REEL_H)

        frames_left = slot_frames - f
        if frames_left <= FADE_FRAMES and next_slot is not None:
            frame_b = get_transform(frame_b_base, 0.0, next_kf, REEL_W, REEL_H)
            t_trans = 1.0 - (frames_left / FADE_FRAMES)
            frame_out = apply_transition(frame_a, frame_b, t_trans,
                                         REEL_W, REEL_H, name=slot.transition_out)
        else:
            frame_out = frame_a

        frame_out = apply_grade(frame_out, theme.color_grade)

        if slot.caption:
            frame_out = render_caption(
                frame=frame_out, text=slot.caption,
                style=theme.text_style, t=t_slide,
                w=REEL_W, h=REEL_H,
            )

        write_frame(frame_out)
        frames_written += 1

    # Progress update per slot
    pct = 12 + int(78 * frames_written / max(total, 1))
    _prog(min(pct, 89), f"? Slot {slot_idx+1}/{n_slots} ? {frames_written}/{tot...

finally:
    release_writer()

_prog(90, "? Muxing audio with ffmpeg?")

if audio_path and os.path.isfile(audio_path) and _find_ffmpeg():
    _mux_audio(tmp_silent, audio_path, out_path, duration_s)
    try:
        os.remove(tmp_silent)
    except Exception:
        pass
else:
    if audio_path and not _find_ffmpeg():
        print("[Audio] ffmpeg not installed ? video is silent. See _mux_audio for i...
    try:
        os.replace(tmp_silent, out_path)
    except Exception:
        shutil.copy(tmp_silent, out_path)
```

# TravelBuddy Pro - Comprehensive Engineering Report

```
return os.path.isfile(out_path) and os.path.getsize(out_path) > 10_000

def _find_ffmpeg() -> Optional[str]:
    """Find ffmpeg binary - checks PATH first, then common Windows install locations."""
    # 1. Check PATH (works on Linux / Mac / Windows if added to PATH)
    path_find = shutil.which("ffmpeg")
    if path_find:
        return path_find
    # 2. Common Windows install locations (including our custom install)
    windows_paths = [
        r"C:\Users\mansi\ffmpeg\bin\ffmpeg.exe",    # installed by TravelBuddy setup
        r"C:\ffmpeg\bin\ffmpeg.exe",
        r"C:\Program Files\ffmpeg\bin\ffmpeg.exe",
        r"C:\Program Files (x86)\ffmpeg\bin\ffmpeg.exe",
        r"C:\tools\ffmpeg\bin\ffmpeg.exe",
        r"C:\ProgramData\chocolatey\bin\ffmpeg.exe",
    ]
    for p in windows_paths:
        if os.path.isfile(p):
            return p
    return None

def _mux_audio(video_path: str, audio_path: str, out_path: str, duration_s: int = DURAT...
    """Mux audio into video via ffmpeg. Shows clear warning if ffmpeg not found."""
    ffmpeg = _find_ffmpeg()
    if not ffmpeg:
        print("[Audio] WARNING: ffmpeg not found. Video will be silent.")
        print("[Audio] Install ffmpeg: https://ffmpeg.org/download.html")
        print("[Audio] Or: winget install ffmpeg / choco install ffmpeg")
        shutil.copy(video_path, out_path)
        return

    cmd = [
        ffmpeg, "-y",
        "-i", video_path,
        "-stream_loop", "-1", # Loop audio if shorter than video
        "-i", audio_path,
        "-c:v", "copy",
        "-c:a", "aac", "-b:a", "192k",
        "-t", str(duration_s),
        "-map", "0:v:0",      # Use video from first input
        "-map", "1:a:0",      # Use audio from second input
        out_path,
    ]
    print(f"[Audio] Muxing: {ffmpeg} ...")
    try:
        result = subprocess.run(cmd, capture_output=True, timeout=300)
        if result.returncode != 0:
            print(f"[Audio] ffmpeg error:\n{result.stderr.decode(errors='ignore')}")
            shutil.copy(video_path, out_path)
        else:
            print("[Audio] Mux complete.")
    except Exception as e:
        print(f"[Audio] Mux exception: {e}")
        shutil.copy(video_path, out_path)

# ?????????????????????????????????????????????????????????????????????????????????????
```

# TravelBuddy Pro - Comprehensive Engineering Report

```
# BACKGROUND JOB RUNNER
# ??????????????????????????????????????????????????????????????????????????????????????

def _run_job(
    job_id: str,
    image_paths: List[str],
    captions: List[str],
    destination: str,
    theme_name: str,
    audio_path: Optional[str],
    lrc_content: Optional[str],
    duration_s: int,
    precomputed metas: Optional[List[PhotoMeta]] = None,
):
    try:
        _set_job(job_id, status="running", progress=0, message="? Starting cinematic pi...)

        import concurrent.futures

        theme = get_theme(theme_name)

        _set_job(job_id, progress=2, message="? Analyzing audio and images concurrently...")

        with concurrent.futures.ThreadPoolExecutor(max_workers=3) as executor:
            if audio_path and os.path.isfile(audio_path):
                beatmap_future = executor.submit(analyze_audio, audio_path, float(durati...)
            else:
                beatmap_future = executor.submit(_synthetic_beatmap, float(duration_s))

            if not precomputed_metas:
                photos_meta_future = executor.submit(analyze_photos, image_paths)

            # Parallel resolve
            beatmap = beatmap_future.result()

            if precomputed_metas:
                photos_meta = precomputed_metas
            else:
                photos_meta = photos_meta_future.result()

            if not photos_meta:
                _set_job(job_id, status="error", message="No usable photos found.")
                return

            if precomputed_metas:
                # Stage 3 explicitly passed precomputed_metas in the exact order the user c...
                sorted_photos = photos_meta
            else:
                sorted_photos = sort_for_storytelling(photos_meta)

            _set_job(job_id, progress=8, message="? Building 5-section EDL timeline?")
            from backend.services.caption_renderer import parse_lrc, get_lyric_at
            lyrics = parse_lrc(lrc_content) if lrc_content else []

            slots = build_timeline(
                photos=sorted_photos,
                beatmap=beatmap,
                theme=theme,
                captions=captions or [],
                force_keep_all=(precomputed_metas is not None),
            )
```

# TravelBuddy Pro - Comprehensive Engineering Report

```
if lyrics:
    for slot in slots:
        lyric = get_lyric_at(lyrics, slot.start_s)
        if lyric and not slot.caption:
            slot.caption = lyric

safe = destination.replace(" ", "_").replace("/", "_")[:40]
fname = f"cinematic_{safe}_{theme_name}_{job_id[:8]}.mp4"
out = str(OUTPUT_DIR / fname)

_set_job(job_id, progress=10, message=f"? Rendering {len(slots)} slots at 720p/..."

success = _render_cinematic(
    slots=slots, theme=theme,
    out_path=out, audio_path=audio_path,
    job_id=job_id, duration_s=duration_s,
)

if success:
    # ?? Cloud Sync (Supabase) ?????????????????????????????????????????????
    remote_url = None
    try:
        from backend.services.supabase_storage import supabase_storage
        if supabase_storage.enabled:
            _set_job(job_id, message="?? Syncing to Supabase Cloud...")
            remote_url = supabase_storage.upload_file(out, fname)
    except Exception as e:
        print(f"?? Supabase Sync failed: {e}")

    _set_job(
        job_id,
        status="done",
        progress=100,
        message=(
            f"? Cinematic reel ready ? {len(sorted_photos)} photos, "
            f"theme '{theme.display_name}', BPM {beatmap.bpm:.0f}, "
            f"'{beat-synced}' if audio_path else 'synthetic beat grid'}. "
            f"'(Uploaded to Oracle Cloud)' if remote_url else ''"
        ),
        video_url=remote_url or f"/data/generated_videos/{fname}",
        beat_map={
            "bpm": round(beatmap.bpm, 1),
            "sections": beatmap.sections,
            "cut_count": len(beatmap.cut_points),
        },
        photo_count=len(sorted_photos),
        theme=theme_name,
    )
else:
    _set_job(job_id, status="error",
            message="Render failed. Ensure OpenCV and Pillow are installed.")

except Exception as exc:
    import traceback
    _set_job(job_id, status="error", message=f"Pipeline error: {exc}",
            traceback=traceback.format_exc())
```

```
# ?????????????????????????????????????????????????????????????????????
# PUBLIC API
# ?????????????????????????????????????????????????????????????????????
```

# TravelBuddy Pro - Comprehensive Engineering Report

```
def generate_cinematic_video(
    image_paths: List[str],
    captions: Optional[List[str]] = None,
    destination: str = "trip",
    theme_name: str = "cinematic",
    audio_path: Optional[str] = None,
    lrc_content: Optional[str] = None,
    duration_s: int = DURATION_S,
    precomputed metas: Optional[List[PhotoMeta]] = None,
) -> dict:
    """
    Kick off a background cinematic render job.
    Returns immediately with a job_id for polling via /video/status/{job_id}.
    """
    _ensure_output_dir()
    job_id = str(uuid.uuid4())

    _set_job(job_id, status="queued", progress=0,
             message="Queued ? starting shortly?",
             photo_count=len(image_paths), theme=theme_name)

    t = threading.Thread(
        target=_run_job,
        args=(job_id, image_paths, captions or [], destination,
              theme_name, audio_path, lrc_content, duration_s, precomputed_metas),
        daemon=True,
    )
    t.start()

    return {
        "status": "queued",
        "job_id": job_id,
        "engine": "cinematic",
        "photo_count": len(image_paths),
        "theme": theme_name,
        "message": "Render started in background. Poll /video/status/{job_id} for p...",
        "poll_url": f"/video/status/{job_id}",
    }
```

# TravelBuddy Pro - Comprehensive Engineering Report

Source File: backend\\services\\media\_intelligence.py

[illegible]

# TravelBuddy Pro - Comprehensive Engineering Report

```
@dataclass
class PhotoMeta:
    """Rich metadata for a single photo ? v2 expanded schema."""
    path: str
    width: int = 0
    height: int = 0
    orientation: str = "landscape"

    # Technical quality
    shot_type: str = "wide"
    blur_score: float = 0.0 # Laplacian variance (higher = sharper)
    exposure_score: float = 0.0 # 0?1 (1 = perfectly exposed)
    brightness: float = 0.0 # mean luminance 0?255
    contrast: float = 0.0 # std-dev of luminance
    noise_score: float = 1.0 # 0?1 (1 = clean, 0 = very noisy)
    entropy: float = 0.0 # Shannon entropy (texture richness)

    # Semantic quality
    face_score: float = 0.0 # 0?10 (higher = prominent face)
    face_count: int = 0 # number of detected faces

    # Travel aesthetics
    color_vibrancy: float = 0.0 # 0?1 composite color score
    golden_hour: float = 0.0 # 0?1 warm/sunset tones
    sky_presence: float = 0.0 # 0?1 blue sky detected
    saturation_score: float = 0.0 # 0?1 colorfulness

    # Composition
    composition_score: float = 0.0 # 0?1 composite composition
    rule_of_thirds: float = 0.0 # 0?1 ROT adherence
    symmetry: float = 0.0 # 0?1 bilateral symmetry

    # Final
    overall_quality: float = 0.0
    ai_insight: str = "" # weighted composite 0?1
    story_position: str = "middle"
    sort_index: int = 0
    is_duplicate: bool = False
    duplicate_of: str = "" # Path of the superior version

    # v3 additions (Improvements 4 & 7)
    scene_type: str = "general" # portrait | landscape | architecture | gen...
    score_breakdown: dict = field(default_factory=dict) # per-metric normalized ...

    @property
    def is_usable(self) -> bool:
        """
        Usability bar v3 (Improvement 3) ? stricter thresholds.
        Resolution gate runs before analysis; these check computed scores.
        """
        return (
            self.blur_score > 25.0 # raised from 10 ? reject soft/motion-blurred
            and self.exposure_score > 0.25 # raised from 0.20 ? reject crushed/blown s...
            and self.noise_score > 0.30 # raised from 0.25 ? reject heavy grain
            and min(self.width, self.height) >= 720 # resolution gate
        )

    @property
    def is_travel_hero(self) -> bool:
        """
        Is this a "hero" travel shot? Used to promote openers/closers.
```

# TravelBuddy Pro - Comprehensive Engineering Report

```

Criteria: vibrant landscape OR golden-hour shot OR prominent face.
"""
return (
    self.golden_hour > 0.4
    or self.sky_presence > 0.5
    or (self.face_score > 2.0 and self.overall_quality > 0.6)
    or (self.color_vibrancy > 0.55 and self.composition_score > 0.5)
)

# ?? Low-level image stats ??????????????????????????????????????????????????????????????

def _compute_brightness(image: np.ndarray) -> float:
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    return float(np.mean(gray))

def _compute_contrast(image: np.ndarray) -> float:
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    return float(np.std(gray))

def _detect_orientation(w: int, h: int) -> str:
    ratio = w / max(h, 1)
    if ratio > 1.15:
        return "landscape"
    if ratio < 0.87:
        return "portrait"
    return "square"

def _classify_shot_type_v2(image: np.ndarray, face_score: float, face_count: int) -> str:
    """
    Improved shot type classifier v2.

    v1 problem: used only face_score > 0.5 for 'medium' ? too aggressive.
    A group photo with 4 small faces has face_score ? 0.4 ? wrongly classified as 'wide'.

    v2 logic:
    close    ? dominant face (>= 1 face covers >8% of image area)
    medium   ? faces present (>= 1 face, or face_count > 0)
    wide     ? high color/spatial variance ? scenic landscape
    detail   ? low variance, tight texture/macro ? street detail, food, architecture
    """
    h_img, w_img = image.shape[:2]
    image_area = h_img * w_img

    # Face-based classification
    if face_count > 0:
        if face_score > 4.0:
            return "close"
        if face_score > 0.3 or face_count >= 2:
            return "medium"

    # For non-face shots, use spatial and color analysis
    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
    sat_std = float(np.std(hsv[:, :, 1]))
    hue_std = float(np.std(hsv[:, :, 0]))

    # High-frequency detail analysis
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

```

# TravelBuddy Pro - Comprehensive Engineering Report

```
laplacian = cv2.Laplacian(gray, cv2.CV_64F)
local_var = float(np.std(laplacian))

# Sky/landscape detection (wide shot proxy)
upper_sat = float(np.mean(hsv[:h_img//3, :, 1]))
lower_var = float(np.std(gray[h_img//2:, :]))

if sat_std > 30 or hue_std > 25:
    # High color variance across image ? scenic wide shot
    return "wide"

if local_var > 80 and sat_std < 20:
    # High-frequency texture with low color variance ? detail/macro/architecture
    return "detail"

if upper_sat < 30 and lower_var > 15:
    # Low-saturation sky (grey/cloudy) with varied ground ? wide landscape
    return "wide"

# Default
return "wide" if sat_std > 15 else "detail"

def _count_faces(image: np.ndarray) -> int:
    """Count raw number of faces detected."""
    try:
        import cv2
        gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
        face_cascade_path = cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'
        cascade = cv2.CascadeClassifier(face_cascade_path)
        if cascade.empty():
            return 0
        faces = cascade.detectMultiScale(gray, scaleFactor=1.3, minNeighbors=4, minSize...)
        return len(faces)
    except Exception:
        return 0

def _compute_exposure_v2(image: np.ndarray) -> float:
    """
    Improved exposure scorer v2.

    v1 problem: only checks pixels < 20 (black) and > 235 (white).
    This incorrectly penalizes intentional silhouettes and creative low-key shots.

    v2 approach: use a bell-curve around the ideal mid-tone (mean luminance 100±160).
    Also separately checks histogram distribution for "good spread" (good dynamic range).
    """
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY).astype(np.float32)
    mean_lum = float(gray.mean())

    # Bell curve centered at ideal 128 (mid-grey) with falloff
    # Score peaks at mean_lum=128, acceptable range 80±180
    ideal = 128.0
    sigma = 55.0
    bell_score = np.exp(-0.5 * ((mean_lum - ideal) / sigma) ** 2)

    # Dynamic range check: a good photo uses the full histogram range
    hist = cv2.calcHist([gray.astype(np.uint8)], [0], None, [256], [0, 256]).flatten()
    hist_norm = hist / (hist.sum() + 1e-6)
    # Check spread: variance of histogram should be reasonably high
```

# TravelBuddy Pro - Comprehensive Engineering Report

```
bins = np.arange(256)
hist_mean = float((bins * hist_norm).sum())
hist_var = float(((bins - hist_mean)**2 * hist_norm).sum())
hist_std = np.sqrt(hist_var)
# Good photos: hist std 40±80
dynamic_range = min(1.0, hist_std / 65.0)

# Penalize extremely clipped pixels (>15% black or >15% white)
over_exposed = float((gray > 240).sum()) / gray.size
under_exposed = float((gray < 15).sum()) / gray.size
clip_penalty = max(0.0, 1.0 - (over_exposed + under_exposed) * 3.0)

exposure = 0.45 * float(bell_score) + 0.35 * dynamic_range + 0.20 * clip_penalty
return round(min(1.0, max(0.0, exposure)), 4)

def _composite_quality_v2(
    blur: float,
    exposure: float,
    contrast: float,
    face_score: float,
    color_vibrancy: float,
    composition: float,
    noise: float,
    entropy: float,
    scene: str = 'general',
    # pre-normalized values injected by analyze_photos after batch_normalize
    blur_norm_batch: float = None,
    exposure_norm_batch: float = None,
    noise_norm_batch: float = None,
    color_norm_batch: float = None,
) -> float:
    """
    Improvement 5: Hybrid geometric-mean + weighted-aesthetic composite.

    Critical metrics (sharpness, exposure, noise) use geometric mean?one bad
    value pulls the whole score down, preventing great-color/blurry photos
    from scoring well. Aesthetic metrics use weighted sum per scene profile.
    """
    import math

    # ?? Normalizations ?????????????????????????????????????????????????????????????
    # Use batch-normalized values when available (Improvement 2), else fall back
    # to the original per-photo log-scale / linear normalizations.
    blur_n = blur_norm_batch if blur_norm_batch is not None else min(1.0, np.log10(blur) + 1.0)
    exposure_n = exposure_norm_batch if exposure_norm_batch is not None else exposure
    noise_n = noise_norm_batch if noise_norm_batch is not None else noise
    color_n = color_norm_batch if color_norm_batch is not None else color_vibrancy

    contrast_n = min(1.0, contrast / 75.0)
    face_n = min(1.0, face_score / 8.0)
    entropy_n = min(1.0, entropy / 7.5)

    # ?? Scene-adaptive weights (Improvement 4) ?????????????????????????????????????
    w = WEIGHT_PROFILES.get(scene, WEIGHT_PROFILES['general'])

    # ?? Geometric mean of critical metrics (Improvement 5) ?????????????????????????
    critical = [
        max(blur_n, 1e-6),
        max(exposure_n, 1e-6),
        max(noise_n, 1e-6),
```

# TravelBuddy Pro - Comprehensive Engineering Report

```

]
geo_mean = math.pow(math.prod(critical), 1 / 3)

# ?? Weighted aesthetic sum ?????????????????????????????????????????????????????????????
aesthetic = (
    color_n * w['color'] +
    composition * w['composition'] +
    face_n * w['face'] +
    entropy_n * w['texture']
)

base = 0.60 * geo_mean + 0.40 * aesthetic

# ?? Special modifiers (preserved from v2) ?????????????????????????????????????????????
if face_score > 2.5 and blur_n > 0.6 and exposure_n > 0.55:
    base = min(1.0, base + 0.05) # face bonus
if noise_n < 0.4 and exposure_n < 0.35:
    base *= 0.75 # dark noise penalty

return round(float(np.clip(base, 0.0, 1.0)), 4)

# ?? v3 Helpers (Improvements 2, 4, 6) ?????????????????????????????????????????????
def batch_normalize(values: list) -> list:
    """
    Improvement 2: Percentile-based batch normalization.
    Scales values to [0,1] using the 5th-95th percentile of the batch,
    making scores adaptive to each user's photo library.
    """
    arr = np.array(values, dtype=float)
    if len(arr) < 2:
        return [float(np.clip(v, 0.0, 1.0)) for v in values]
    p5, p95 = np.percentile(arr, [5, 95])
    scale = p95 - p5 + 1e-8
    return [float(np.clip((v - p5) / scale, 0.0, 1.0)) for v in values]

# Improvement 4: Scene-adaptive weight profiles
WEIGHT_PROFILES = {
    'portrait': {'sharpness': 0.40, 'color': 0.20, 'exposure': 0.15,
                 'composition': 0.10, 'face': 0.10, 'noise': 0.03, 'texture': 0.02},
    'landscape': {'sharpness': 0.28, 'color': 0.28, 'exposure': 0.18,
                  'composition': 0.15, 'face': 0.00, 'noise': 0.06, 'texture': 0.05},
    'architecture': {'sharpness': 0.30, 'color': 0.15, 'exposure': 0.15,
                     'composition': 0.25, 'face': 0.03, 'noise': 0.05, 'texture': 0.07},
    'general': {'sharpness': 0.35, 'color': 0.20, 'exposure': 0.15,
                'composition': 0.12, 'face': 0.08, 'noise': 0.05, 'texture': 0.05},
}

def classify_scene(img_bgr: np.ndarray, face_score: float = 0.0) -> str:
    """
    Improvement 4: Lightweight heuristic scene classifier (no ML model).
    Returns: 'portrait' | 'landscape' | 'architecture' | 'general'
    """
    if img_bgr is None or img_bgr.size == 0:
        return 'general'
    try:
        gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
        h, w = gray.shape

```

# TravelBuddy Pro - Comprehensive Engineering Report

```
# Portrait: any face detected
if face_score > 0.0:
    return 'portrait'

# Landscape: significant blue sky in upper third
hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)
sky_region = hsv[:, :, :].copy()
blue_mask = cv2.inRange(sky_region, (90, 40, 40), (130, 255, 255))
if blue_mask.sum() > sky_region.size * 0.3:
    return 'landscape'

# Architecture: many straight lines via Hough
edges = cv2.Canny(gray, 50, 150)
lines = cv2.HoughLinesP(edges, 1, np.pi / 180, 80, minLineLength=100)
if lines is not None and len(lines) > 15:
    return 'architecture'

return 'general'
except Exception:
    return 'general'
```

```
def temporal_sample(photos: List[str], n_buckets: int = 10) -> List[str]:
    """
    Improvement 6: Time-bucket sampling.
    Groups photos by file mtime into n_buckets equal buckets.
    From each bucket, keeps only the top-scoring photo.
    Prevents a single golden-hour burst from dominating the reel.
    """
    if len(photos) <= n_buckets:
        return photos

    def _get_time(p):
        try:
            return os.path.getmtime(p.path)
        except Exception:
            return 0.0

    photos_sorted = sorted(photos, key=_get_time)
    bucket_size = max(1, len(photos_sorted) // n_buckets)
    result = []
    for i in range(n_buckets):
        bucket = photos_sorted[i * bucket_size:(i + 1) * bucket_size]
        if bucket:
            result.append(max(bucket, key=lambda p: p.overall_quality))
    # Include any remainder photos not covered by the last full bucket
    remainder_start = n_buckets * bucket_size
    if remainder_start < len(photos_sorted):
        remainder = photos_sorted[remainder_start:]
        result.append(max(remainder, key=lambda p: p.overall_quality))
    return result
```

```
# ?? Public API ??????????????????????????????????????????????????????????????????????
```

```
def analyze_photos(image_paths: List[str]) -> List[PhotoMeta]:
    """
    Analyze every photo and return a list of PhotoMeta objects.
    Photos that fail to load are excluded.
```

# TravelBuddy Pro - Comprehensive Engineering Report

```
v3 pipeline order:
1. Parallel per-photo analysis (_analyze_single)
2. Perceptual deduplication (dHash)
3. Improvement 2: Batch normalization of blur/exposure/noise/color across all surviving photos ? adaptive to the user's library.
4. Recompute overall_quality with batch-normalized values + scene weights.
5. Improvement 6: Temporal sampling ? enforce time diversity.
6. Narrative sorting (sort_for_storytelling)
"""
import concurrent.futures
from models.vision.deduplication import remove_duplicates

raw_results: List[dict] = []

# ?? Step 1: Parallel analysis ??????????????????????????????????????????????????????????????
with concurrent.futures.ThreadPoolExecutor(max_workers=16) as executor:
    future_to_path = {executor.submit(_analyze_single, path): path for path in imag...}
    for future in concurrent.futures.as_completed(future_to_path):
        try:
            meta = future.result()
            if meta is not None:
                raw_results.append({
                    "path": meta.path,
                    "blur": meta.blur_score,
                    "sharpness": meta.blur_score,
                    "face": meta.face_score,
                    "exposure": meta.exposure_score,
                    "contrast": meta.contrast,
                    "entropy": meta.entropy,
                    "meta": meta,
                })
        except Exception as e:
            print(f"[Vision v3] Error analyzing photo: {e}")

if not raw_results:
    return []

# ?? Step 2: Deduplication ??????????????????????????????????????????????????????????????
unique_results = remove_duplicates(raw_results, threshold=6)
unique_paths = {d["path"] for d in unique_results}

all metas: List[PhotoMeta] = [d["meta"] for d in raw_results]
usable_metas: List[PhotoMeta] = []
for meta in all_metas:
    if meta.path not in unique_paths:
        meta.is_duplicate = True
        meta.score_breakdown["gate_passed"] = False
        meta.score_breakdown["reject_reason"] = "duplicate"
    elif meta.is_usable:
        usable_metas.append(meta)
    else:
        # Mark rejection reason for explainability (Improvement 7)
        if meta.blur_score <= 25.0:
            reason = "blur_too_low"
        elif meta.exposure_score <= 0.25:
            reason = "exposure_too_low"
        elif meta.noise_score <= 0.30:
            reason = "noise_too_high"
        else:
            reason = "resolution_too_low"
        meta.score_breakdown["gate_passed"] = False
```

# TravelBuddy Pro - Comprehensive Engineering Report

```
meta.score_breakdown["reject_reason"] = reason

if not usable metas:
    return all_metas # Return everything (all failed gates) so caller can report

# ?? Step 3: Improvement 2 ? Batch normalize critical metrics ??????????????
blur_raw = [m.blur_score for m in usable_metas]
exp_raw = [m.exposure_score for m in usable_metas]
noise_raw = [m.noise_score for m in usable_metas]
color_raw = [m.color_vibrancy for m in usable_metas]

blur_norm_batch = batch_normalize(blur_raw)
exp_norm_batch = batch_normalize(exp_raw)
noise_norm_batch = batch_normalize(noise_raw)
color_norm_batch = batch_normalize(color_raw)

# ?? Step 4: Recompute overall_quality with batch norms + scene weights ???
for i, meta in enumerate(usable_metas):
    quality = _composite_quality_v2(
        blur=meta.blur_score, exposure=meta.exposure_score,
        contrast=meta.contrast, face_score=meta.face_score,
        color_vibrancy=meta.color_vibrancy,
        composition=meta.composition_score,
        noise=meta.noise_score, entropy=meta.entropy,
        scene=meta.scene_type,
        blur_norm_batch=blur_norm_batch[i],
        exposure_norm_batch=exp_norm_batch[i],
        noise_norm_batch=noise_norm_batch[i],
        color_norm_batch=color_norm_batch[i],
    )
    meta.overall_quality = quality
    # Update breakdown with batch-normalized values (Improvement 7)
    meta.score_breakdown.update({
        "sharpness": round(blur_norm_batch[i], 3),
        "exposure": round(exp_norm_batch[i], 3),
        "noise": round(noise_norm_batch[i], 3),
        "color": round(color_norm_batch[i], 3),
        "final": round(quality, 4),
    })

# ?? Step 5: Improvement 6 ? Temporal sampling ?????????????????????????????
temporally_sampled = temporal_sample(usable_metas, n_buckets=10)

# ?? Step 6: Narrative sort ?????????????????????????????????????????????
# (sort_for_storytelling is called by the reel endpoint, not here,
# to preserve API compatibility ? analyze_photos returns scored+sampled list)
return all_metas # full list; caller uses .is_usable and .is_duplicate flags

def sort_for_storytelling(photos: List[PhotoMeta]) -> List[PhotoMeta]:
    """
    Order photos for the cinematic reel.

    FIX ? Chronological-first ordering:
    ?????????????????????????????????????????????????????????????????
    OLD approach: sorted by quality score ? photos jumped around in time.
    Result: a face taken at 9 AM could appear after a sunset at 7 PM.

    NEW approach:
    1. Sort all photos by file mtime (= phone camera capture time).
    This preserves the natural story order the user experienced.
```

# TravelBuddy Pro - Comprehensive Engineering Report

```
2. Pick the BEST quality photo as the opener (swap it to front).
   This hooks the viewer immediately, even if it was taken mid-trip.
3. Pick the best face-shot as the closer (swap it to end).
   Emotional farewell, but everything between stays in time order.
????????????????????????????????????????????????????????????????
"""
if not photos:
    return photos

# ?? 1. Sort chronologically by file mtime ??????????????????????????????
def _mtime(p: PhotoMeta) -> float:
    try:
        return os.path.getmtime(p.path)
    except Exception:
        return 0.0

chrono = sorted(photos, key=_mtime)

n = len(chrono)
if n <= 2:
    for i, p in enumerate(chrono):
        p.story_position = "opener" if i == 0 else "closer"
        p.sort_index = i
    return chrono

# ?? 2. Pick opener: highest overall_quality (often landscape/golden hour) ?
best_idx = max(range(n), key=lambda i: chrono[i].overall_quality + chrono[i].color_...
opener_photo = chrono[best_idx]
remaining = [p for i, p in enumerate(chrono) if i != best_idx]

# ?? 3. Pick closer: best face shot for emotional farewell ??????????????????
face_shots = [(i, p) for i, p in enumerate(remaining) if p.face_score > 1.0]
if face_shots:
    closer_idx, closer_photo = max(face_shots, key=lambda x: x[1].face_score + x[1]...
else:
    # No face shots ? use last photo chronologically as closer
    closer_idx = len(remaining) - 1
    closer_photo = remaining[closer_idx]
middle = [p for i, p in enumerate(remaining) if i != closer_idx]

# ?? 4. Keep middle in chronological order ? NO re-sorting ??????????????????
# Middle photos stay exactly as the user experienced them on the trip.

# ?? 5. Assign story positions for timeline weighting ??????????????????????
opener_photo.story_position = "opener"
closer_photo.story_position = "closer"
total_mid = len(middle)
for i, p in enumerate(middle):
    t = i / max(total_mid - 1, 1)
    if t < 0.25:
        p.story_position = "early"
    elif t < 0.60:
        p.story_position = "middle"
    elif t < 0.80:
        p.story_position = "peak"
    else:
        p.story_position = "middle"

final = [opener_photo] + middle + [closer_photo]
for i, p in enumerate(final):
    p.sort_index = i
```

# TravelBuddy Pro - Comprehensive Engineering Report

```
    return final

def _enforce_shot_diversity(photos: List[PhotoMeta]) -> List[PhotoMeta]:
    """
    Re-order to prevent more than 2 consecutive photos of the same shot_type.
    Uses a simple greedy approach: if the next photo would create a run of 3,
    insert the next different type instead.
    """
    if len(photos) < 3:
        return photos

    result = []
    remaining = list(photos)

    while remaining:
        # Check if adding remaining[0] creates a run of 3
        if (len(result) >= 2 and
            result[-1].shot_type == result[-2].shot_type == remaining[0].shot_type):
            # Find next different shot type
            inserted = False
            for i in range(1, len(remaining)):
                if remaining[i].shot_type != remaining[0].shot_type:
                    result.append(remaining.pop(i))
                    inserted = True
                    break
            if not inserted:
                result.append(remaining.pop(0))
        else:
            result.append(remaining.pop(0))

    return result

# ?? Internal helpers ?????????????????????????????????????????????????????????????

def _get_cache_path(image_path: str) -> Path:
    cache_dir = Path("data/cache/vision")
    cache_dir.mkdir(parents=True, exist_ok=True)
    mtime = os.path.getmtime(image_path)
    # v2 cache key includes version to avoid stale v1 caches
    base_hash = hashlib.sha256(f"v2_{image_path}_{mtime}".encode()).hexdigest()[:16]
    return cache_dir / f"{Path(image_path).name}_{base_hash}.json"

def _analyze_single(path: str) -> Optional[PhotoMeta]:
    """
    Analyze one photo. Returns None if the file cannot be opened.
    Loads image once and passes to all scorers ? no redundant I/O.
    """
    if not os.path.isfile(path):
        return None

    # ?? Improvement 3: Resolution gate (before any heavy processing) ??????????
    try:
        import PIL.Image as _PILImg
        with _PILImg.open(path) as _im:
            _w, _h = _im.size
            if min(_w, _h) < 720:
                meta = PhotoMeta(path=path, width=_w, height=_h)
```

# TravelBuddy Pro - Comprehensive Engineering Report

```
        meta.score_breakdown = {
            "scene": "unknown", "sharpness": 0.0, "exposure": 0.0,
            "noise": 0.0, "color": 0.0, "composition": 0.0,
            "face": 0.0, "texture": 0.0, "final": 0.0,
            "gate_passed": False, "reject_reason": "resolution_too_low",
        }
        return meta # is_usable will also be False via width/height check
except Exception:
    pass # If PIL can't read dimensions, proceed normally

# Check cache (v2 cache key)
cache_path = _get_cache_path(path)
if cache_path.exists():
    try:
        with open(cache_path, "r") as f:
            data = json.load(f)
            # Must have new v3 fields to be valid
            if "color_vibrancy" in data and "composition_score" in data and "scene_...
                return PhotoMeta(**data)
    except Exception:
        pass # Fall through to fresh analysis

image = load_image_for_analysis(path)
if image is None:
    return None

h_img, w_img = image.shape[:2]
orientation = _detect_orientation(w_img, h_img)

# ?? Technical scores ??????????????????????????????????????????????????????????????
blur      = calculate_blur_score(image=image)
entropy   = calculate_entropy_score(image=image)
noise     = calculate_noise_score(image=image)
brightness = _compute_brightness(image)
contrast  = _compute_contrast(image)
exposure  = _compute_exposure_v2(image)

# ?? Semantic scores ??????????????????????????????????????????????????????????????
face_score = detect_faces(image=image)
face_count = _count_faces(image)

# ?? Aesthetic scores ??????????????????????????????????????????????????????????????
color_data = calculate_color_vibrancy(image)
comp_data  = calculate_composition_score(image)

# ?? Improvement 4: Scene classification ??????????????????????????????????????????
scene = classify_scene(image, face_score=face_score)

# ?? Shot type (v2 improved) ??????????????????????????????????????????????????????
shot_type = _classify_shot_type_v2(image, face_score, face_count)

# ?? Composite quality (v3 formula ? batch norms filled in later) ??????????????
# Note: batch-normalized values are None here; analyze_photos() will
# recompute overall_quality after batch normalization is available.
quality = _composite_quality_v2(
    blur=blur, exposure=exposure, contrast=contrast,
    face_score=face_score, color_vibrancy=color_data["composite"],
    composition=comp_data["composite"], noise=noise, entropy=entropy,
    scene=scene,
)
```

# TravelBuddy Pro - Comprehensive Engineering Report

```
# ?? Improvement 7: Per-metric normalized scores for explainability ???????
blur_n    = min(1.0, float(np.log1p(blur) / np.log1p(3000.0)))
face_n    = min(1.0, face_score / 8.0)
entropy_n = min(1.0, entropy / 7.5)
score_breakdown = {
    "scene":      scene,
    "sharpness":  round(blur_n, 3),
    "exposure":   round(exposure, 3),
    "noise":      round(noise, 3),
    "color":      round(color_data["composite"], 3),
    "composition": round(comp_data["composite"], 3),
    "face":       round(face_n, 3),
    "texture":    round(entropy_n, 3),
    "final":      round(quality, 4),
    "gate_passed": True,
}

# ?? Generate AI Insight ?????????????????????????????????????????????
insights = []
if color_data["golden_hour"] > 0.6: insights.append("Stunning golden hour light")
elif color_data["sky_presence"] > 0.6: insights.append("Beautiful open sky")
if face_score > 5.0: insights.append("Prominent, well-captured face")
elif face_count >= 3: insights.append(f"Great group moment with {face_count} people")
if comp_data["rule_of_thirds"] > 0.7: insights.append("Excellent cinematic framing")
if color_data["composite"] > 0.7: insights.append("Vibrant, travel-ready colors")
if quality > 0.85: insights.append("Professional-grade technical quality")
ai_insight = " ? ".join(insights) if insights else "Clean, sharp travel shot"

meta = PhotoMeta(
    path=path, width=w_img, height=h_img,
    orientation=orientation, shot_type=shot_type,
    blur_score=round(blur, 2), exposure_score=round(exposure, 4),
    brightness=round(brightness, 2), contrast=round(contrast, 2),
    noise_score=round(noise, 4), entropy=round(entropy, 4),
    face_score=round(face_score, 4), face_count=face_count,
    color_vibrancy=round(color_data["composite"], 4),
    golden_hour=round(color_data["golden_hour"], 4),
    sky_presence=round(color_data["sky_presence"], 4),
    saturation_score=round(color_data["saturation_score"], 4),
    composition_score=round(comp_data["composite"], 4),
    rule_of_thirds=round(comp_data["rule_of_thirds"], 4),
    symmetry=round(comp_data["symmetry"], 4),
    overall_quality=quality,
    ai_insight=ai_insight,
    scene_type=scene,
    score_breakdown=score_breakdown,
)

# Save to cache
try:
    from dataclasses import asdict
    with open(cache_path, "w") as f:
        json.dump(asdict(meta), f)
except Exception as e:
    print(f"[Vision v3] Cache write error (non-fatal): {e}")

return meta
```

# TravelBuddy Pro - Comprehensive Engineering Report

**Source File:** backend\\services\\color\_grader.py

```

backend/services/color_grader.py ? Cinematic Color Grading Engine

Phase 3: Apply theme-defined color grading to each frame's NumPy array.

Operations (applied in order):
1. Brightness adjustment          (linear scale on all channels)
2. Contrast adjustment            (center-point contrast)
3. Saturation adjustment          (HSV S-channel scale)
4. Warm / cool tint               (per-channel RGB additive offset)
5. Vignette overlay               (radial gradient darkening toward edges)

All operations are pure NumPy/OpenCV ? 100% offline, no LUT files required.
"""

from __future__ import annotations

import cv2
import numpy as np

from backend.config.themes import ColorGrade

def apply_grade(frame: np.ndarray, grade: ColorGrade) -> np.ndarray:
    """
    Apply cinematic color grade to a single BGR frame (uint8, HxWx3).

    Args:
        frame: NumPy array, dtype uint8, BGR colour order (OpenCV standard).
        grade: ColorGrade settings from the active Theme.

    Returns:
        Graded frame, same shape and dtype.
    """
    out = frame.astype(np.float32)

    # ?? 1. Brightness ?????????????????????????????????????????????????????????????????
    if grade.brightness != 1.0:
        out *= grade.brightness

    # ?? 2. Contrast ?????????????????????????????????????????????????????????????????
    if grade.contrast != 1.0:
        # Centre-point contrast: scale around midpoint (128)
        out = (out - 128.0) * grade.contrast + 128.0

    # ?? 3. Saturation ???????????????????????????????????????????????????????????????
    if grade.saturation != 1.0:
        # Convert to HSV, scale S channel, convert back
        u8 = np.clip(out, 0, 255).astype(np.uint8)
        hsv = cv2.cvtColor(u8, cv2.COLOR_BGR2HSV).astype(np.float32)
        hsv[:, :, 1] = np.clip(hsv[:, :, 1] * grade.saturation, 0, 255)
        out = cv2.cvtColor(hsv.astype(np.uint8), cv2.COLOR_HSV2BGR).astype(np.float32)

    # ?? 4. Warm / cool tint (RGB additive offset) ?????????????????????????????????
    r_add, g_add, b_add = grade.warm_tint # tuple is (R, G, B) ? we store as BGR
    if r_add or g_add or b_add:
        # OpenCV stores as BGR
        out[:, :, 2] += r_add # red channel

```

# TravelBuddy Pro - Comprehensive Engineering Report

```
    out[:, :, 1] += g_add    # green channel
    out[:, :, 0] += b_add    # blue channel

# Clamp before vignette
out = np.clip(out, 0, 255)

# ?? 5. Vignette ?????????????????????????????????????????????????????????????
if grade.vignette > 0.0:
    out = _apply_vignette(out, grade.vignette)

return np.clip(out, 0, 255).astype(np.uint8)

# ?? Vignette generator (cached per resolution) ???????????????????????????????

_vignette_cache: dict[tuple, np.ndarray] = {}

def _apply_vignette(frame: np.ndarray, strength: float) -> np.ndarray:
    """
    Darken corners with a smooth radial gradient vignette.

    Strength 0 = none, 1 = very heavy (edges almost black).
    """
    h, w = frame.shape[:2]
    cache_key = (h, w, round(strength, 3))

    if cache_key not in _vignette_cache:
        # Build normalised distance map from centre
        cx, cy = w / 2.0, h / 2.0
        y_coords = np.linspace(0, h - 1, h)
        x_coords = np.linspace(0, w - 1, w)
        xv, yv = np.meshgrid(x_coords, y_coords)
        dist = np.sqrt(((xv - cx) / cx) ** 2 + ((yv - cy) / cy) ** 2)
        # Smooth cosine falloff
        vignette = np.cos(np.clip(dist * 1.2, 0, 1) * np.pi / 2) ** 2
        # Apply strength ? at strength=1 the corners reach 0
        vignette = 1.0 - (1.0 - vignette) * strength
        mask = np.stack([vignette, vignette, vignette], axis=2).astype(np.float32)
        _vignette_cache[cache_key] = mask

    mask = _vignette_cache[cache_key]
    return (frame * mask).astype(np.float32)

# ?? Convenience: normalize exposure before grading ?????????????????????????????

def normalize_exposure(frame: np.ndarray, target_mean: float = 128.0) -> np.ndarray:
    """
    Linearly scale frame brightness so its mean luminance meets target_mean.
    This counters over- or under-exposed photo inconsistencies before grading.

    Args:
        frame:          BGR uint8 frame.
        target_mean:    Desired mean luminance (default 128 for mid-tone).

    Returns:
        Exposure-normalized frame (uint8).
    """
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    mean = float(np.mean(gray))
```

# TravelBuddy Pro - Comprehensive Engineering Report

```
if mean < 5.0:
    return frame    # black frame ? skip
scale = target_mean / mean
# Only correct if significant deviation (?20%)
if 0.8 <= scale <= 1.25:
    return frame
normalized = np.clip(frame.astype(np.float32) * scale, 0, 255).astype(np.uint8)
return normalized
```

# TravelBuddy Pro - Comprehensive Engineering Report

## Source File: scripts\release.py

```
"""
scripts/release.py ? TravelBuddy One-Command Release Pipeline
????????????????????????????????????????????????????????????

USAGE (from the project root ? c:\\Users\\mansi\\TravelBuddy):
    python scripts/release.py

WHAT IT DOES (fully automatic, zero manual steps):
    1. Reads current build number from mobile/pubspec.yaml
    2. Bumps the build number by +1 (1.0.0+5 ? 1.0.0+6)
    3. Runs: flutter build apk --release (inside mobile/)
    4. Deletes old APK from Google Drive (the previous version)
    5. Uploads new APK to Google Drive ? makes it public
    6. Updates version.json with new build number + Drive APK URL
    7. Updates backend/api/health.py with the new build number
    8. Prints the public APK URL so you can verify it

When users next open the old APK:
    ? Splash screen calls GET /version
    ? Backend returns the new build number
    ? App shows "Update Available" dialog with download button
    ? User taps ? browser downloads the new APK from Drive
"""

import os
import re
import sys
import json
import subprocess
from pathlib import Path

# Force UTF-8 output on Windows (avoids UnicodeEncodeError in PowerShell)
import io
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8', errors='replace')

# ?? Google Drive Auth ?????????????????????????????????????????????????????????????
from google.auth.transport.requests import Request
from google.oauth2.credentials import Credentials
from google_auth_oauthlib.flow import InstalledAppFlow
from googleapiclient.discovery import build
from googleapiclient.http import MediaFileUpload

# ?????????????????????????????????????????????????????????????????????????????
# CONFIG ? edit once, never touch again
# ?????????????????????????????????????????????????????????????????????????????
ROOT = Path(__file__).resolve().parent.parent # c:\\Users\\mansi\\TravelBuddy
MOBILE_DIR = ROOT / "mobile"
PUBSPEC = MOBILE_DIR / "pubspec.yaml"
APK_SRC = MOBILE_DIR / "build" / "app" / "outputs" / "flutter-apk" / "app-release..."
VERSION_JSON = ROOT / "version.json"
HEALTH_PY = ROOT / "backend" / "api" / "health.py"
CLIENT_SECRET = ROOT / "client_secret.json"
TOKEN_FILE = ROOT / "token.json"
DRIVE_FOLDER = "1sU2AWrjt4awLYBzcP6f97N-qMpAfr-Sq" # your existing Drive folder
APK_FILENAME = "app-release.apk"

SCOPES = [
    "https://www.googleapis.com/auth/drive.file",
```

# TravelBuddy Pro - Comprehensive Engineering Report

```
"https://www.googleapis.com/auth/drive.metadata.readonly",
]

SEPARATOR = "=" * 60

# ?????????????????????????????????????????????????????????????????????
# -----
# STEP 0: Pretty logger
# -----
def log(msg: str, icon: str = "?"):
    print(f"\n{icon} {msg}")

def ok(msg: str):
    print(f"    [OK] {msg}")

def fail(msg: str):
    print(f"\n    [FATAL] {msg}")
    sys.exit(1)

# -----
# STEP 1: Bump version in pubspec.yaml
# -----
def bump_version() -> tuple[str, int, str]:
    """Returns (new_version_string, new_build_number, original_pubspec_text)."""
    log("Reading current version from pubspec.yaml", "?")
    original_text = PUBSPEC.read_text(encoding="utf-8")

    match = re.search(r"^version:\s+(\d+\.\d+\.\d+)\s+(\d+)", original_text, re.MULTILINE)
    if not match:
        fail("Could not find 'version: x.y.z+N' in pubspec.yaml")

    semver      = match.group(1)
    old_build   = int(match.group(2))
    new_build   = old_build + 1
    new_ver_str = f"{semver}+{new_build}"

    updated = original_text.replace(
        f"version: {semver}+{old_build}",
        f"version: {semver}+{new_build}",
    )
    PUBSPEC.write_text(updated, encoding="utf-8")

    ok(f"Version bumped: {semver}+{old_build} ? {new_ver_str}")
    return new_ver_str, new_build, original_text

# ?????????????????????????????????????????????????????????????????????
# STEP 2: Build release APK
# ?????????????????????????????????????????????????????????????????????
def build_apk(original_pubspec_text: str):
    """
    Runs flutter build apk --release.
    On failure, rolls back pubspec.yaml to avoid a phantom version bump.
    Uses shell=True so Python finds flutter.bat on Windows PATH.
    """
    log("Building Flutter release APK (this takes 2?4 minutes)?", "?")
    result = subprocess.run(
        "flutter build apk --release",
        cwd=str(MOBILE_DIR),
```

# TravelBuddy Pro - Comprehensive Engineering Report

```

shell=True, # Required on Windows: flutter is a .bat file
text=True,
)
if result.returncode != 0:
    # Rollback the version bump so retrying starts from the same number
    PUBSPEC.write_text(original_pubspec_text, encoding="utf-8")
    fail("flutter build apk failed. pubspec.yaml has been restored. Fix the error a...")
if not APK_SRC.exists():
    PUBSPEC.write_text(original_pubspec_text, encoding="utf-8")
    fail(f"APK not found after build: {APK_SRC} ? pubspec.yaml restored.")
size_mb = APK_SRC.stat().st_size / (1024 * 1024)
ok(f"APK built successfully ({size_mb:.1f} MB)")

# ?????????????????????????????????????????????????????????????????????????????????????
# STEP 3: Authenticate with Google Drive
# ?????????????????????????????????????????????????????????????????????????????????????
def get_drive_service():
    log("Authenticating with Google Drive?", "?")
    if not CLIENT_SECRET.exists():
        fail(f"client_secret.json not found at {CLIENT_SECRET}")

    creds = None
    if TOKEN_FILE.exists():
        creds = Credentials.from_authorized_user_file(str(TOKEN_FILE), SCOPES)

    if not creds or not creds.valid:
        if creds and creds.expired and creds.refresh_token:
            creds.refresh(Request())
            ok("Token refreshed from cache.")
        else:
            # First-time: opens browser once to authorize, then saves token.json
            flow = InstalledAppFlow.from_client_secrets_file(str(CLIENT_SECRET), SCOPES)
            creds = flow.run_local_server(port=0)
            ok("Browser auth completed.")
            TOKEN_FILE.write_text(creds.to_json())

    ok("Google Drive authenticated.")
    return build("drive", "v3", credentials=creds)

# ?????????????????????????????????????????????????????????????????????????????????????
# STEP 4: Delete old APK from Drive, upload new one
# ?????????????????????????????????????????????????????????????????????????????????????
def upload_apk_to_drive(service) -> str:
    """Returns the public direct-download URL of the newly uploaded APK."""
    log("Uploading APK to Google Drive (57 MB ? may take a few minutes)...", "")

    # Delete any existing APK with the same name in the folder
    existing = service.files().list(
        q=f"name='{APK_FILENAME}' and '{DRIVE_FOLDER}' in parents and trashed=false",
        fields="files(id, name)",
    ).execute().get("files", [])

    for f in existing:
        service.files().delete(fileId=f["id"]).execute()
        print(f"    [DEL] Deleted old APK from Drive (id={f['id']})")

    # Upload new APK with progress reporting
    metadata = {"name": APK_FILENAME, "parents": [DRIVE_FOLDER]}
    media = MediaFileUpload(

```

# TravelBuddy Pro - Comprehensive Engineering Report

```

        str(APK_SRC),
        mimetype="application/vnd.android.package-archive",
        resumable=True,
        chunksize=5 * 1024 * 1024,    # 5 MB chunks
    )
    request = service.files().create(body=metadata, media_body=media, fields="id")

    apk_id = None
    response = None
    while response is None:
        status, response = request.next_chunk()
        if status:
            pct = int(status.progress() * 100)
            print(f"    Uploading... {pct}%", end="\r", flush=True)

    apk_id = response["id"]
    print()    # newline after progress

    # Make public
    service.permissions().create(
        fileId=apk_id,
        body={"type": "anyone", "role": "reader"},
    ).execute()

    apk_url = f"https://drive.google.com/uc?export=download&id={apk_id}"
    print(f"    [OK] APK uploaded and made public.")
    print(f"    [OK] APK Download URL: {apk_url}")
    return apk_url

# ?????????????????????????????????????????????????????????????????????????????????????
# STEP 5: Update version.json (the local config file)
# ?????????????????????????????????????????????????????????????????????????????????????
def update_version_json(new_build: int, apk_url: str):
    log("Updating version.json?", "?")

    # Read backend_url from existing version.json if available
    existing_url = "http://10.196.231.234:8000"
    if VERSION_JSON.exists():
        try:
            existing = json.loads(VERSION_JSON.read_text())
            existing_url = existing.get("backend_url", existing_url)
        except Exception:
            pass

    data = {
        "latest_version": new_build,
        "backend_url":    existing_url,
        "apk_url":        apk_url,
        "update_message": f"TravelBuddy v{new_build} is here! Update now for the latest..."
    }
    VERSION_JSON.write_text(json.dumps(data, indent=4))
    ok(f"version.json updated with build={new_build}")

# ?????????????????????????????????????????????????????????????????????????????????????
# STEP 6: Patch backend/api/health.py with new build number + APK URL
# ?????????????????????????????????????????????????????????????????????????????????????
def patch_health_py(new_build: int, apk_url: str):
    log("Patching backend/api/health.py?", "?")
    text = HEALTH_PY.read_text(encoding="utf-8")

```

# TravelBuddy Pro - Comprehensive Engineering Report

```
# Replace LATEST_VERSION
text = re.sub(
    r"LATEST_VERSION\s*=\s*\.+",
    f'LATEST_VERSION = {new_build}',
    text,
)
# Replace DRIVE_FILE_ID (extract ID from apk_url)
drive_file_id = apk_url.split("id=")[-1]
text = re.sub(
    r"DRIVE_FILE_ID\s*=\s*\.+",
    f'DRIVE_FILE_ID = "{drive_file_id}"',
    text,
)
HEALTH_PY.write_text(text, encoding="utf-8")
ok(f"health.py ? LATEST_VERSION = {new_build}, DRIVE_FILE_ID = {drive_file_id}")

# ?????????????????????????????????????????????????????????????????????????????????????
# MAIN
# ?????????????????????????????????????????????????????????????????????????????????????
if __name__ == "__main__":
    print(f"\n{SEPARATOR}")
    print("  TravelBuddy Automated Release Pipeline")
    print(SEPARATOR)

    new_ver_str, new_build, original_pubspec = bump_version()
    build_apk(original_pubspec)
    service = get_drive_service()
    apk_url = upload_apk_to_drive(service)
    update_version_json(new_build, apk_url)
    patch_health_py(new_build, apk_url)

    print(f"\n{SEPARATOR}")
    print(f"  ? RELEASE v{new_ver_str} COMPLETE!")
    print(f"{SEPARATOR}")
    print(f"  Build Number : {new_build}")
    print(f"  APK URL      : {apk_url}")
    print(f"\n  ? Old APK users will see the update dialog on next launch.")
    print(f"  ? No manual steps required.")
    print(f"  ? Restart your FastAPI backend to serve the new /version data.")
    print(f"\n  To restart backend: uvicorn backend.main:app --host 0.0.0.0 --port 800...")
    print(SEPARATOR + "\n")
```